

Comparison Concepts

Date: 2018-09-24

Author: David Stone (davidmstone@google.com, david@doublewise.net)

Audience: LEWG

Proposal

This paper proposes allowing anything which is currently specified as accepting a `BinaryPredicate` to also accept a function that returns a comparison category implicitly convertible to `weak_equality`. Anything which is currently specified as accepting a `Compare` should also accept a function that returns a comparison category implicitly convertible to `weak_ordering`.

This paper also proposes a general library requirement that if `a <=> b` is valid, it is consistent with all other comparison operators. In other words, for every comparison operator (`==`, `!=`, `<`, `<=`, `>`, and `>=`) and for every value of `a` and `b`, `(a @ b) == (a <=> b @ 0)`. In particular, any function that accepts a type meeting the `LessThanComparable` concept should be able to call `a <=> b` instead of `a < b` if `a <=> b` is well-formed. Whether an implementation calls `<=>` or any of the 6 traditional comparison operators is not considered an observable difference.

This paper does not recommend changing the definitions of the comparable type concepts (such as `LessThanComparable`) to require `operator<=>`. `operator<=>` is a valid implementation strategy to satisfy these concepts, but should not be required, as all that we care about for these concepts is the direct comparisons. This ensures backward compatibility with old user-defined types.

Motivation

This is especially important in the case of `Compare` to maximize efficiency. We would like users to be able to pass in function objects that are three-way-comparison-aware. Right now, `std::map<std::string, T, Compare>` has to make two passes over each `string` it compares if that `string` does not compare less. The `operator<=>` proposal gives us a more efficient way to do things (guaranteed single pass), so we should take advantage of it where possible. This paper does not propose changing the default template argument for `map`, but this paper does allow users to create something more efficient.

For `BinaryPredicate`, the issue has more to do with consistency. If a user has written a function returning a comparison category, we know that they intend it to be used to compare objects. It is a frustrating and incomplete user experience to not be able to use that function object consistently. This is especially true if the user is trying to perform multiple operations on the same type of elements. For instance, they may want to write a comparison function that can be passed to both `sort` and `unique`. This is not possible right now, because `sort` requires a `Compare`, but `unique` requires a `BinaryPredicate`. By integrating comparison categories into these concepts, we allow a single function object to meet both requirements.

Background

There are two sets of concepts in the standard library that relate to comparisons: concepts that constrain comparison function objects, and concepts that constrain comparable types.

Comparison Function Objects

`BinaryPredicate`

A BinaryPredicate must accept two arguments and return a value contextually convertible to bool, representing whether those two objects are equal. BinaryPredicate is used in the following places in the standard:

- default_searcher
- boyer_moore_searcher
- boyer_moore_horspool_searcher
- forward_list::unique
- list::unique
- find_end
- find_first_of
- adjacent_find
- mismatch
- equal
- is_permutation
- search
- search_n
- unique
- unique_copy

Compare.

A Compare must accept two arguments and return a bool representing whether the first argument is less than the second, and it must provide a strict weak order ($a < b$ implies $!(b < a)$, and equality is determined by $!(a < b)$ and $!(b < a)$). Compare is used in the following places in the standard:

- AssociativeContainer (map, multimap, set, multiset)
- forward_list::merge
- forward_list::sort
- list::merge
- list::sort
- priority_queue
- sort
- stable_sort
- partial_sort
- partial_sort_copy
- is_sorted
- is_sorted_until
- nth_element
- lower_bound
- upper_bound
- equal_range
- binary_search
- merge
- inplace_merge
- includes
- set_union
- set_intersection
- set_difference
- set_symmetric_difference
- push_heap
- pop_heap
- make_heap
- sort_heap

- `is_heap`
- `is_heap_until`
- `min`
- `max`
- `minmax`
- `min_element`
- `max_element`
- `minmax_element`
- `clamp`
- `lexicographical_compare`
- `next_permutation`
- `prev_permutation`

Comparable Type Concepts

- `EqualityComparable`: requires only `a == b`.
- `LessThanComparable`: requires only that `a < b` provides a strict weak ordering.
- `NullablePointer`: inherits `EqualityComparable` and turns it into `weak_equality`, additionally requiring `a != b`. Additionally requires heterogeneous `==` and `!=` (`weak_equality`) with `nullptr_t`.
- `InputIterator`: inherits `EqualityComparable` and turns it into `weak_equality`, additionally requiring `a != b`.
- `Allocator`: requires `weak_equality` (`a == b` and `a != b`), requires heterogeneous comparison with the equivalent `Allocator` for other types
- "bitmask type": `strong_equality` (it might be a `bitset`, which supports only equality, it might be integer or enum, which supports ordering, but this will naturally fall out, no wording changes are needed here)
- We redundantly specify `weak_ordering` for `TrivialClock::rep`, `TrivialClock::duration`, and `TrivialClock::time_point`, but that is implied because we know they are an arithmetic type, a `chrono::duration`, and a `chrono::time_point`, respectively.